

Baseline → Feature: What Changed to Add the "Like" Counter

File	Layer	Baseline (before)	Feature (after)
<code>ansible/files/schema.sql</code>	Schema (on-prem)	<code>guestbook</code> table has no <code>likes</code> column.	Adds <code>ALTER TABLE guestbook ADD COLUMN IF NOT EXISTS likes INTEGER NOT NULL DEFAULT 0; —</code> idempotent, existing rows default to <code>0</code> .
<code>ansible/site.yml</code>	Provisioning (on-prem)	No task guarantees the <code>likes</code> column exists on hosts provisioned before the feature.	New task <code>Ensure the reactions (likes) column exists on the guestbook table</code> runs the same idempotent <code>ALTER TABLE ... ADD COLUMN IF NOT EXISTS likes ...</code> via <code>become_user: postgres</code> , so already-provisioned DBs gain the column on re-run.
<code>cdk/app.py</code>	API Gateway (AWS)	<code>guestbook</code> resource only has <code>GET</code> and <code>POST</code> methods.	Adds a nested resource <code>guestbook/{id}/like</code> with a <code>POST</code> method — purely additive, so existing <code>GET/POST</code> on <code>guestbook</code> are untouched.
<code>cdk/lambda/handler.py</code>	Lambda logic	<code>_list_entries/_add_entry</code> select/return <code>id, name, message, created_at</code> ; no route handles <code>/like</code> .	Queries now also select/return <code>likes</code> ; new <code>_like_entry()</code> runs <code>UPDATE guestbook SET likes = likes + 1 ... RETURNING ...</code> ; <code>handler()</code> routes any resource ending in <code>/like</code> with <code>POST</code> to it, validating a numeric <code>id</code> (400) and a missing entry (404).
<code>frontend/app.js</code>	Frontend	Each entry renders <code>who, when, message</code> only.	Each entry also renders a <code>♥ <count> <button</code>

File	Layer	Baseline (before)	Feature (after)
			<code>class="like"></code> that calls <code>likeEntry()</code> , which <code>POSTs</code> to <code>/guestbook/{id}/like</code> and updates the button text from the response.
<code>frontend/styles.css</code>	Frontend styling	<code>.entries li</code> has no flex layout; no <code>.like</code> button styles.	<code>.entries li</code> becomes a column flex container; new <code>.entries button.like</code> (+ <code>:hover</code> , <code>:disabled</code>) styles render the pill-shaped like button.

Net effect

- **Baseline:** the guestbook lists and accepts entries, but there is no way to react to one — no DB column, no API route, no UI control.
- **Feature:** every entry has a persistent, on-prem-stored like counter that can be incremented from the UI via a new, additive API route.

How it ships

Every change is additive (new column with `DEFAULT`, new API resource/method, new Lambda branch, new UI element) — no existing schema, route, or resource is modified or removed. `cdk deploy` therefore produces a changeset with `UPDATE_COMPLETE` and no replacement, and the Ansible playbook re-applies idempotently (`ADD COLUMN IF NOT EXISTS`, `GRANT` is naturally idempotent), preserving the already-deployed VPC, CloudFront distribution, DB secret value, and existing rows (which simply default to `likes = 0`). See `DEMO_SCRIPT.md` for the full narrated walkthrough.